describe

排序方法	平均情况	最好情况	最差情况	辅助空间	稳定性
冒泡排序	O(n^2)	O(n)	O(n^2)	O(1)	稳定
选择排序	O(n^2)	O(n^2)	O(n^2)	O(1)	(不)稳定
插入排序	O(n^2)	O(n)	O(n^2)	O(1)	稳定
希尔排序	O($nlog^2n$)~O(n^2)	O($nlog^2n$)	O(n^2)	O(1)	不稳定
堆排序	O(nlogn)	O(nlogn)	O(nlogn)	O(1)	不稳定
归并排序	O(nlogn)	O(nlogn)	O(nlogn)	O(n)	稳定
快速排序	O(nlogn)	O(nlogn)	O(n^2)	O(logn)~O(n)	不稳定

1.插入insert sort



Time: O(n^2) Space: O(n)

We sort a series from left to right. And for each step, the left side is the series already sorted, and on the right side is the unsorted series. So in each step we pick out the first unsorted element, and compare it with the sorted series, then insert it into the right position. So we repeat this process until the whole series is sorted.

(就地排序不用额外的辅助数组Sort in place, no extra auxiliary array)

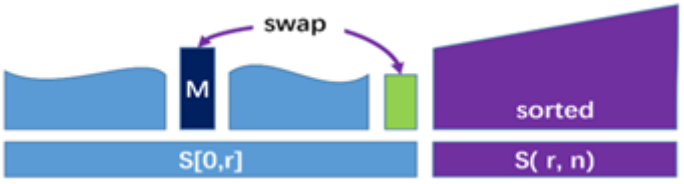
Best: sorted 1->N O(n)

Worst: N->1 decreasing series O(n^2)

Vector: comparing O(n^2) + shifting O(n^2)

List: comparing O(n^2) + shifting O(n)

2.选择select sort



Time: O(n^2) Space: O(n)

We can sort from right to left. For each step, the left side is the unsorted series, and the right side is sorted series. Then we select the maximal element from the unsorted series, and put it on the left of the sorted series. This way after n times repeat we can get a sorted series.

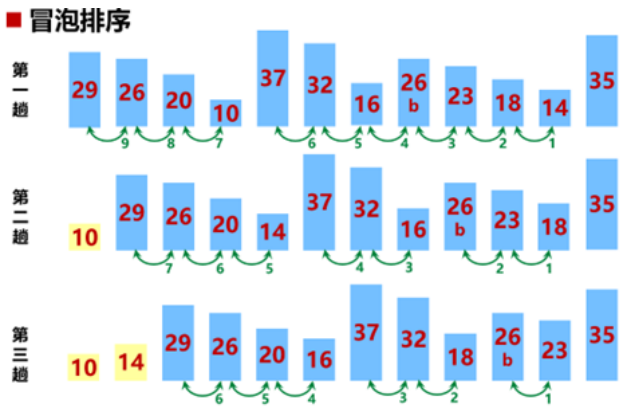
(就地排序不用额外的辅助数组Sort in place, no extra auxiliary array)

Best /Worst: O(n^2) (n^2 to choose max)

Vector: comparing O(n^2)+exchanging O(n) (swap maximum and the last unsorted element)

List: comparing O(n^2) + exchanging O(n)

3.冒泡 bubble sort



Time: O(n^2) Space: O(n)

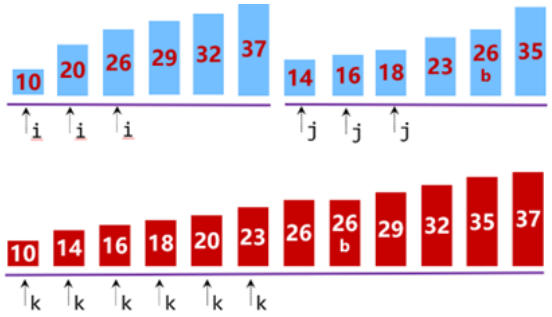
In this algorithm we conduct at most n loops of sorting. In each loop we sort from right to left and follow a rule: if the number bigger than the left one next to it, then remain its position, and continue with the next number on the left. If the number is smaller than the one on the left, exchange the 2 numbers and therefore this number shift left for a step, repeat until this number encounter a smaller number on the left.

This is one single loop, and we just repeat this process until the whole series is sorted. After at most n loops, we can get a sorted series.

Best: sorted 1->N we only need to conduct one loop to find the series sorted: O(n)

Worst: N->1 decreasing series n loops O(n^2)

4.归并排序 merge sort 不是就地排序（not sort in place, need an extra auxiliary array)



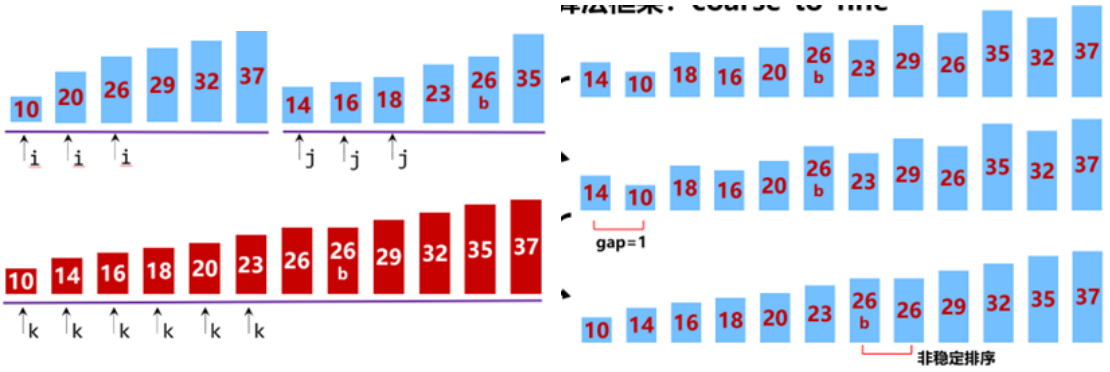
Time: O(nlogn) Space: O(n)+ O(n)

We just use divide-and-conquer method. For a series whose length is n, we divide it into 2 halves, and for each half, we again apply merge sort to it. So in a recursive way, the problem is simplified to many simple sorting problems(n*simplest: len=1, O(1) to sort). After getting 2 sorted subseries, we need to merge them together. For each merging, let's assume the len of subseries==x, we define an auxiliary array with length 2x, and we compare the first elements of the 2 subseries, then pop out the smaller one into the auxiliary array, repeat the process for 2x times. This process is O(2x)

Best/worst O(nlogn)

n=2^m, proof there are m layers of recursion. In the x th layer of recursion, there are 2^(m-x) merge, and the length of subseries is x, so in this layer the total time complexity is 2^(m-x)*2^x=2^m, and actually this is the same for each layer. So the time complexity for merge sort is O(m*2^m)=O(nlogn) x从1到m

5.Shell sort　(not stable)



This algorithm based on idea coarse-to-fine. Firstly, we resample the original series with gap length x. in other words we pick out one per x elements in original series, so we can get x group of series in total. Then conduct insert sort for each group. Then we decrease the x(e.g. x/2), repeat such process, until the gap length is 1. Finally we get a sorted series.

The key idea is that Sorting at coarse scale can reduce more inversion pairs.

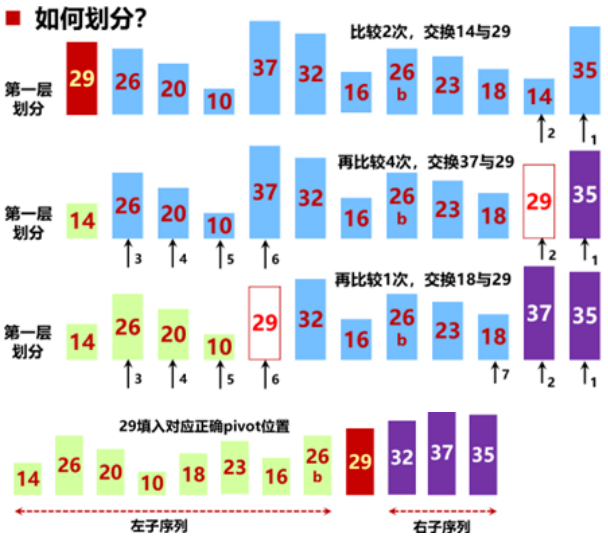
最坏情况为O(n4/3)，平均情况为O(n7/6)

It can be proofed that the worst case is no more than O(n^2)

6.quick sort

跟其他复杂度同为nlogn的算法相比，快速排序的常数系数是最小的，速度是最快的，而且需要的额外空间为logn可以忽略。不足是运气最坏的时候会变成n^2。N小的时候反而慢，适合n大的，无序性高的。

递归排序慢一点，且需要更多的存储空间.



Time: avg O(nlogn); best O(nlogn); worst O(n^2)

Space(space cost for calling the recursion stack): avg O(logn); best O(logn); worst O(n)

The quick sort also uses divide-and-conquer. For one series, we first choose the first element as its pivot. And use exchanging method to put it to the middle. In other words, its left side are all smaller than it while the right side are bigger than it(this will take O(x)). Then we use recursive way, to conduct quick sort in both left and right subseries.

How to exchange? If we consider the first element as pivot, then start from right side, compare the right side element with the pivot. If pivot is smaller, then from right to left continue the comparing. If pivot is bigger, then exchange pivot and this value. Now the pivot is on the right side, the element on its right are all bigger than pivot. So we need to compare from left side. If pivot is bigger, shift right and continue. If pivot is smaller, exchange the number with pivot to put the pivot on the left side. Repeat until all elements have been compared with pivot. And we can get pivot in the middle.

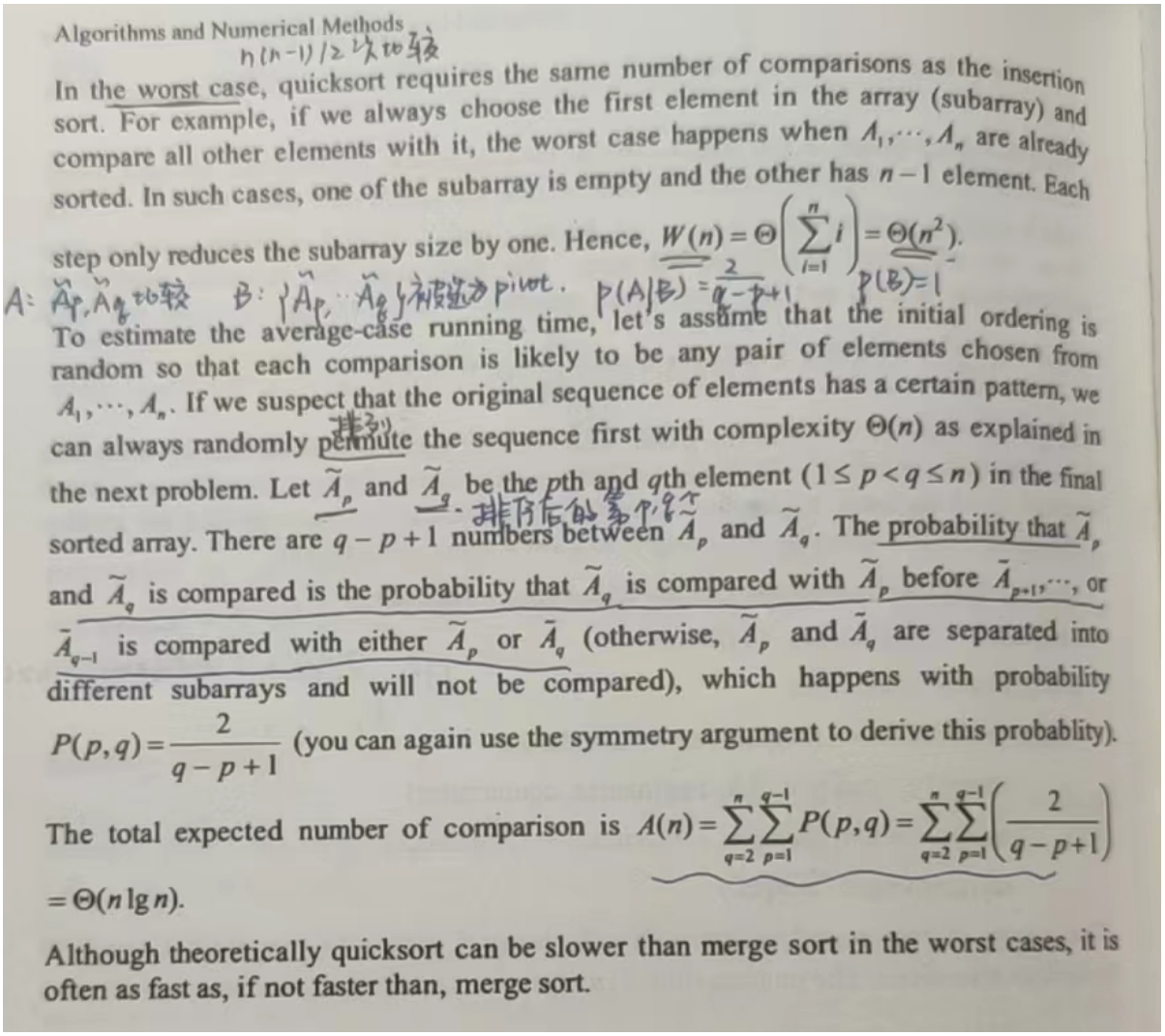
Best 每次pivot对半分（median number， if we draw the recursion process represented by a binary tree, we may find this situation has the lowest height）

O(n)+ 2O(n/2)+ 4O(n/4)+...=O(nlog2n)

Worst 有序 O(n^2)

Avg T(n)=n+1+1/n * Σ（T(i-1)+T(n+1)）à O((n+1)* 2ln2 * log2n)

Proof:



Algorithms and Numerical Methods

In the worst case, quicksort requires the same number of comparisons as the insertion sort. For example, if we always choose the first element in the array (subarray) and compare all other elements with it, the worst case happens when $A_1, \cdots, A_n$ are already sorted. In such cases, one of the subarray is empty and the other has $n-1$ element. Each step only reduces the subarray size by one. Hence, $W(n) = \Theta\left(\sum_{i=1}^{n} i\right) = \Theta(n^2)$.

To estimate the average-case running time, let's assume that the initial ordering is random so that each comparison is likely to be any pair of elements chosen from $A_1, \cdots, A_n$. If we suspect that the original sequence of elements has a certain pattern, we can always randomly permute the sequence first with complexity $\Theta(n)$ as explained in the next problem. Let $\tilde{A}_p$ and $\tilde{A}_q$ be the pth and qth element ($1 \le p < q \le n$) in the final sorted array. There are $q-p+1$ numbers between $\tilde{A}_p$ and $\tilde{A}_q$. The probability that $\tilde{A}_p$ and $\tilde{A}_q$ is compared is the probability that $\tilde{A}_q$ is compared with $\tilde{A}_p$ before $\tilde{A}_{p+1}, \cdots,$ or $\tilde{A}_{q-1}$ is compared with either $\tilde{A}_p$ or $\tilde{A}_q$ (otherwise, $\tilde{A}_p$ and $\tilde{A}_q$ are separated into different subarrays and will not be compared), which happens with probability $P(p,q) = \frac{2}{q-p+1}$ (you can again use the symmetry argument to derive this probablity).

The total expected number of comparison is $A(n) = \sum_{q=2}^{n}\sum_{p=1}^{q-1} P(p,q) = \sum_{q=2}^{n}\sum_{p=1}^{q-1}\left(\frac{2}{q-p+1}\right)$

$= \Theta(n \lg n)$.

Although theoretically quicksort can be slower than merge sort in the worst cases, it is often as fast as, if not faster than, merge sort.

7.heap sort

O(nlogn) O(1)

Heap is actually priority queue. To get a heap, we can use Floyd algorithm to construct it, whose complexity is O(n)

And for each step we remove the top element of the heap, and take O(logn) to maintain the heap, then we can get a sorted series after we repeat n times.

8.bucket sort (sort by key)

给定[0,M)范围内的n个整数

Space O(M)

time创建散列表O(M)，关键码插入O(n)，依次读取关键码O(max(M,n))

(modular hash function)

code

python

```
# insert
def insert_sort(nums):
    # in place    implement based on vector
    if len(nums)<=1:
        return nums
    for i in range(1,len(nums)):
        index_i=0
        for j in range(i):
            if nums[i]>=nums[i-j-1]:
                index_i=i-j
                break
        nums[index_i:i+1]=[nums[i]]+nums[index_i:i]
    return nums

# select
def select_sort(nums):
    # in place    implement based on vector
    if len(nums) <= 1:
        return nums
    for i in range(len(nums)-1):
        MIN = nums[i]
        min_index=i
        for j in range(i+1,len(nums)):
            if nums[j]<MIN:
                MIN=nums[j]
                min_index = j
        nums[min_index] = nums[i]
        nums[i] = MIN
    return nums
```

```python
# bubble
def bubble_sort(nums):
    # in place    implement based on vector
    if len(nums) <= 1:
        return nums
    for i in range(len(nums)):
        all_sorted=True
        for j in range(len(nums)-1,0,-1):
            if nums[j]<nums[j-1]:
                nums[j],nums[j-1]=nums[j-1],nums[j]
                all_sorted=False
        if all_sorted:
            break
    return nums

# merge
def merge(nums1,nums2):
    i=0
    j=0
    temp=[]
    while(i<len(nums1) or j<len(nums2)):
        if i==len(nums1):
            temp.append(nums2[j])
            j += 1
        elif j==len(nums2):
            temp.append(nums1[i])
            i += 1
        elif nums1[i]<=nums2[j]:
            temp.append(nums1[i])
            i += 1
        else:
            temp.append(nums2[j])
            j += 1
    return temp
def merge_sort(nums):
    if len(nums)<=1:
        return nums
    sub_1=merge_sort(nums[:len(nums)//2])
    sub_2=merge_sort(nums[len(nums)//2:])
    return merge(sub_1,sub_2)

# Shell
# 循环很多，但其实复杂度不高
def Shell_sort(nums):
    if len(nums)<=1:
        return nums
    gap=len(nums)//2
    while(gap>=1):
        for i in range(gap): # i表示第几组，对于一个给定的gap一共有gap组
            index_start=i # 第i组第一个元素开始的编号
            index=index_start+gap # index表示每个组内的指针，从组内第二个迭代到最后一个
            # 插入排序
            while index<len(nums):
                j=index  # j表示index指向的元素，在向前比较过程中的指针
                while j>index_start:
```

```
                    if nums[j]<nums[j-gap]:
                        nums[j],nums[j-gap]=nums[j-gap],nums[j]
                        # 不采用整体移位实现插入排序，每次交换法实现插入排序
                        j-=gap
                    else:
                        break
                index+=gap # 进行完一个位置的比较和插入后，向后移一个单位
        gap//=2 # 每次gap减半
    return nums

# quick sort
def quick_sort(nums,start,end):  # 数组就地排序，[start,end)
    if start>=end:
        return
    pivot=nums[start]   # 数组选第一个值为pivot
    i=start # left pointer，一开始pivot的pointer也是i
    j=end-1 # right pointer

    # 主循环
    # 开始时p=i比较右边,当发现nums[j]更小时,交换pivot(即nums[i])和nums[j],并开始比较左边
    while i<j:  # 结束标志是i==j(此时i=j=p)
        while nums[j] >= pivot and i<j:
            j -= 1
        if i < j:   # 发现nums[j]更小时,交换pivot(即nums[i])和nums[j],结果是p=j,并开始
比较左边
            nums[j], nums[i] = nums[i], nums[j]
            i += 1 # 原来左指针位置是换过来的元素，因而左指针右移一位
        while nums[i] <= pivot and i<j:
            i += 1
        if i < j:  # 发现nums[i]更大时,交换pivot(即nums[j])和nums[i],结果是p=i,并开始
比较右边
            nums[i], nums[j] = nums[j], nums[i]
            j -= 1 # 原来右指针位置是换过来的元素，因而右指针右移一位
    quick_sort(nums,start,i) # 递归左子序列
    quick_sort(nums,i+1,end) # 递归右子序列
    return nums[start:end]

# heap      in place
def heap_sort(array):
    first = len(array) // 2 - 1
    # 1.堆构建 Floyd算法（自底向上调整O(n)）
    for start in range(first, -1, -1):
        # 从下到上，从右到左对每个非叶节点进行调整，循环构建成大顶堆
        big_heap(array, start, len(array) - 1)
    # 2.删除+下滤
    for end in range(len(array) - 1, 0, -1):
        # 交换堆顶和堆尾的数据
        array[0], array[end] = array[end], array[0]
        # 重新调整完全二叉树，构造成大顶堆
        big_heap(array, 0, end - 1)
    return array
def big_heap(array, start, end): # 下滤算法：置换+下滤
    # Floyd算法中一次调整的步骤，对一个给定的root节点下滤调整到底
    root = start
    # 左孩子的索引
    child = root * 2 + 1
```

```
    while child <= end:
        # 节点有右子节点，并且右子节点的值大于左子节点，则将child变为右子节点的索引
        if child + 1 <= end and array[child] < array[child + 1]:
            child += 1
        if array[root] < array[child]:
            # 交换节点与子节点中较大者的值
            array[root], array[child] = array[child], array[root]
            # 交换值后，如果存在孙节点，则将root设置为子节点，继续与孙节点进行比较
            root = child
            child = root * 2 + 1
        else:
            break
# backet
nums,result = [19,21,23,14,35,49,37,59,10,16],[]
minv,maxv = min(nums),max(nums) #找出所有元素中的最小值和最大值
countn = [0]*(maxv-minv+1) #算出需要桶的最大个数
for i in nums:
    countn[i-minv] += 1 #桶的编号与对应元素不一致，需要通过计算调整
for i in range(1,len(countn)):
    if countn[i]:
        result += [i+minv]*countn[i]
print(result)
```

c++

```
// insert/////////////////////////////////////////////////////////
void insertSort(int data[], int l, int r) {
    int auxiliary = 0;
    for (int i = l+1; i <= r; i++) {
        if (data[i] < data[i - 1]) {
        auxiliary = data[i];
            int j = i - 1;
            while (j >= l && data[j] > auxiliary) {
                data[j + 1] = data[j];
                j -= 1;
            }
            data[j + 1] = auxiliary;
        }
}
int main(){
    int array[] = { 14, 10, 18, 16, 20, 26, 23, 29, 26, 35, 32, 37 };
    insertSort(array, 0,11);
    for (int i = 0; i < 12; ++i) cout << array[i] << endl;
    return 0;
}


// select/////////////////////////////////////////////////////////
void selectSort(int Data[], int count){
    int i, j, index, temp;
    for (i = count - 1; i > 0; i--){ // 从后往前
        index = i;
        for (j = 0; j < i + 1; j++){ // 遍历前面未排序，选择最大元素
            if (Data[j] > Data[index]) index = j;
        }
```

```
        if (index != i){                // 交换
            temp = Data[i];
        Data[i] = Data[index];
            Data[index] = temp;
        }
    }
}
void main(){
    int array[] = { 14, 10, 18, 16, 20, 26, 23, 29, 26, 35, 32, 37 }, k;
    int len = sizeof(array) / sizeof(int);
    selectSort(array, len);
    for (k = 0; k<len; k++) cout << array[k] << ",";
    cout << endl;
}


// bubble/////////////////////////////////////////////////
void bubbleSort(int data[], int count) {
    int temp;
    int pass = 1; bool exchange = true;             // 从第一趟开始
    while (pass < count && exchange) {
        exchange = false;                   // 某趟是否有交换的标志，初始为无交换
        for (int j = count - 1; j >= pass; j--) //从最后元素开始到第一个未排序元素
            if (data[j - 1]>data[j]) {              //若需要交换则置换元素
                temp = data[j - 1];
                data[j - 1] = data[j];
                data[j] = temp;
                exchange = true;
            }
        pass++;
    }
}
int main(){
    int array[] = { 14, 10, 18, 16, 20, 26, 23, 29, 26, 35, 32, 37 };
    bubbleSort(array, 12);
    for (int i = 0; i < 12; ++i) cout << array[i] << endl;
    return 0;
}

// merge/////////////////////////////////////////////////////
static void merge(int data[], int first, int mid, int last, int sorted[]){
    int i = first, j = mid;
    int k = 0;
    while (i < mid && j < last)
        if (data[i] < data[j])                  // 归并
            sorted[k++] = data[i++];
        else
            sorted[k++] = data[j++];
    while (i < mid) sorted[k++] = data[i++];    // 拷贝两个子序列中的剩余元素
    while (j < last) sorted[k++] = data[j++];
    for (int v = 0; v < k; v++)
        data[first + v] = sorted[v];        // 将排序后结果覆盖原数组
}
static void mergeSort(int data[], int first, int last, int sorted[]){
    if (first + 1 < last){
        int mid = (first + last) / 2;
```

```
        mergeSort(data, first, mid, sorted);
        mergeSort(data, mid, last, sorted);
        merge(data, first, mid, last, sorted);
    }
}

// Shell 希尔排序////////////////////////////////////////////////////////////
void ShellSort(int data[], int count){
    int step = 0;
    int auxiliary = 0;
    for (step = count / 2; step > 0; step /= 2){  // 从数组第step个元素开始
        for (int i = step; i < count; i++){     // 每个元素与自己组内的数据进行直接插入
排序
            if (data[i] < data[i - step]){     // 插入排序的第一次判断
                auxiliary = data[i];           // 需要往前插入，对待插入数据进行缓存
                int j = i - step;
                while (j >= 0 && data[j] > auxiliary){ //对同组前面数据检测，所大则循
环后移
                    data[j + step] = data[j];
                    j -= step;
                }
                data[j + step] = auxiliary;          // 插入数据
            }
         }
     }
}

void main(){
    int array[] = { 14, 10, 18, 16, 20, 26, 23, 29, 26, 35, 32, 37 };
    ShellSort(array, 12);
    for (int i = 0; i < 12; ++i) cout << array[i] << endl;
}

// quick sort////////////////////////////////////////////////////////
void quickSort(int data[], int l, int r){
    if (l< r){
        int pivotL = l, pivotR = r, x = data[l];
        while (pivotL < pivotR){
            while (pivotL<pivotR && data[pivotR]>x) pivotR --;
                     //从右向左找第一个小于x的数
            if (pivotL<pivotR) data[pivotL ++] = data[pivotR];

            while (pivotL<pivotR && data[pivotL]<x) pivotL++;
                    //从左向右找第一个大于等于x的数
            if (pivotL<pivotR) data[pivotR --] = data[pivotL];
        }
        data[pivotL] = x;
        quickSort(data, l, pivotL - 1);   // 递归调用处理左子序列
        quickSort(data, pivotL + 1, r);   // 递归调用处理右子序列
    }
}
void main(){
    int array[] = { 14, 10, 18, 16, 20, 26, 23, 29, 26, 35, 32, 37 }, k;
    int len = sizeof(array) / sizeof(int);
    quickSort(array, 0, len - 1);
    for (k = 0; k<len; k++) cout << array[k] << ",";
```

```
    cout << endl;
}

// heap//////////////////////////////////////////////////////
template <class ElemType>
void MyHeap<ElemType>::sortHeap()
{
    makeHeap();
    while (numCounts > 0)
        delMax(); // 即为每次循环的置换、下滤过程
}
```

二分查找：